

Node Types

Cache	Caching station
ClassSwitch	Class switching
Delay	Infinite server
Fork	Forks jobs
Join	Joins jobs
Logger	Trace logging
Place	Petri net place
Queue	Queueing station
Router	Routing node
Sink	Job departures (open)
Source	Job arrivals (open)
Transition	Petri net transition

Distributions

APH(α , T)	Acyclic PH
Cox2(μ , ϕ)	Two-phase Coxian
Coxian(μ , ϕ)	Coxian
Det(v)	Deterministic
Disabled	None
Erlang(λ , k)	Erlang-k
Exp(λ)	Exponential
Gamma(α , β)	Gamma
HyperExp(p, λ_1 , λ_2)	Hyper-exp
Immediate	Zero service time
Lognormal(μ , σ)	Log-normal
Normal(μ , σ)	Normal
Pareto(α , k)	Pareto
PH(α , T)	Phase-type
Uniform(a, b)	Uniform
Weibull(α , β)	Weibull
Discrete	
Bernoulli(p)	Bernoulli
Binomial(p, n)	Binomial
DiscreteUniform(a, b)	Discr. uniform
Geometric(p)	Geometric
Poisson(λ)	Poisson
Zipf(s, n)	Popularity

Correlated / trace-driven

BMAP(D0...Dk)	Batch MAP
NHPP(bp, λ , cyclic)	Non-homog. Poisson
DMAP(D0, D1)	Discrete MAP
EmpiricalCDF(F)	Empirical CDF
MAP(D0, D1)	MAP
MarkedMAP	Marked MAP
ME(α , T)	Matrix-exp.
CME(mean, order)	Concentr. matrix-exp.
MMPP2(λ_1 , λ_2 , σ_1 , σ_2)	MMPP(2)
RAP(H0, H1)	Rational AP
Replayer(trace)	Trace replay

Scheduling Strategies

DPS	Discriminatory PS
EDD	Earliest Due Date
EDF	Earliest Deadline First
EXT	External (source)
FB	Feedback (LAS)
FCFS	First-Come First-Served
FCFSPI	FCFS Preempt. Resume-idle
FCFSR	FCFS Preemptive Resume
FSP	Fair Sojourn Protocol
GPS	Generalized PS
HOL	Head-of-Line Priority
INF	Infinite Servers
LCFS	Last-Come First-Served
LCFSPI	LCFS Preempt. Resume-idle
LCFSR	LCFS Preemptive Resume
LEPT	Longest Expected PT
LJF	Longest Job First
LPS	Limited PS
LRPT	Longest Remaining PT
OI	Order Independent
PAS	Pass and Swap
POLLING	Polling (switchover)
PS	Processor Sharing
PSJF	Preemptive SJF
SEPT	Shortest Expected PT
SETF	Shortest Elapsed Time First
SIR0	Random
SJF	Shortest Job First
SRPT	Shortest Remaining PT

Java constructors take new, e.g. new Queue(model, "Q", SchedStrategy.FCFS).

Job Classes

```
new ClosedClass(model, "name", N, refStat)
new OpenClass(model, "name")
new SelfLoopingClass(model, "name", N, ref)
new DisabledClass(model, "name")
OpenSignal, ClosedSignal (catastrophes)
```

In Java prefix distribution constructors with new, e.g. new Exp(0.5), or use Exp.fitMean(2.0).

Solvers

- AUTO Auto-select best
- BA Bound analysis (closed)
- CTMC CTMC (exact, small)
- ENV Random environments
- FLD Fluid/mean-field ODEs
- LDES Discrete Event Sim
- LN Layered networks
- MAM Matrix Analytic Methods
- MVA Mean Value Analysis
- NC Normalizing Constant Methods
- SSA Stochastic simulation

Wrappers

- JMT JMT simulator
- LQNS LQNS interface
- QNS LQNS product-form solver

Solver Options

```
new MVA(model, "method", "exact")
new JMT(model, "seed", 123,
"samples", 10000)
new SSA(model, "samples", 5000,
"method", "para")
new LDES(model, new MVA(model)) // warm start
```

Output Metrics

- ArvR Arrival rate
- QLen Mean queue length
- ResidT Residence time (total across visits)
- RespT Response time (per visit)
- Tput Throughput
- Util Server utilization
- QLenVar Queue-length variance
- QLenSCV Queue-length squared coeff. of variation
- QLenSkew Queue-length skewness
- RespTVar Response-time variance (FCFS only)
- RespTSCV Response-time SCV (FCFS only)
- RespTSkew Response-time skewness (FCFS only)

Moments (product form)

```
solver.getMomentTable()
// per class; order 2 by default
solver.getMomentTable(3)
// 1=means 2=+var 3+=skew
solver.getMomentChainTable(3)
// per chain
solver.getMomentStationTable(3)
// per station total
solver.getMomentStationTable(3,"lin")
// approx., for large N
```

order is a set: scalar k = up to k ;
a vector, e.g. [2, 3], = exactly those.
RespT* moments are FCFS-only (NaN elsewhere); exact methods cost $\prod_r (N_r + 1)$, use "lin" beyond that.

Analysis Methods

```
solver.getAvgTable()
solver.getAvgQLen(), getAvgUtil()
solver.getAvgRespT(), getAvgTput()
solver.getAvgSysRespT()
solver.getAvgSysTput()
solver.getAvgQLenChain()
```

Table Methods

- getAvgTable per station
- getAvgChainTable per chain
- getAvgNodeTable per node
- getAvgNodeChainTable node $\times$ chain
- getAvgSysTable system totals

Routing

```
model.link(Network.serialRouting(
n1,n2,n3));
n1.setProbRouting(jobClass, n2, 0.7);
n1.setRouting(jobClass,
RoutingStrategy.RROBIN);
```

Multi-Server & Finite Capacity

```
queue.setNumberOfServers(k);
queue.setCapacity(bufferSize);
```

Class Switching

```
Matrix P = new Matrix("[0.3,0.7; 0.5,0.5]");
ClassSwitch cs =
new ClassSwitch(model, "CS", P);
```

Fork-join Networks

```
Fork fork = new Fork(model, "Fork");
Join join = new Join(model, "Join", fork);
```

Layered Networks

```
LayeredNetwork lqn =
new LayeredNetwork("name");
Processor P = new Processor(lqn,"P",1,
SchedStrategy.PS);
Task T = new Task(lqn,"T",1,
SchedStrategy.REF).on(P);
Entry E = new Entry(lqn,"E").on(T);
Activity A = new Activity(lqn,"A",
new Exp(1)).on(T).boundTo(E);
```

Caching

```
Cache cache = new Cache(model,"Cache",
nItems,cap,ReplacementStrategy.LRU);
cache.setRead(jobClass, new Zipf(1.4, nItems));
// cache results
new MVA(model).getAvgCacheTable(); // hit/miss
new MVA(model).getAvgItemTable(); // per-item
```

ReplacementStrategy

- CLIMB Climb
- FIFO First-in first-out
- HLRU h-LRU
- LRU Least recently used
- QLRU q-LRU
- RR Random replacement
- SFIFO Static FIFO

Model Import/Export

```
model.jsimgView(); // Open in JMT
LineModelIO.save(model,"model.json");
Network model = LineModelIO.load("model.json");
```

Routing Strategies

- JSQ Join shortest queue
- KCHOICES Power of k choices
- PROB Probabilistic routing
- RAND Random selection
- RROBIN Round-robin
- WROBIN Weighted round-robin

Stochastic Petri Nets

```
Place p = new Place(model, "P1");
Transition t = new Transition(model, "T1");
int m = t.addMode("Model1");
t.setNumberOfServers(m, Integer.MAX_VALUE);
t.setDistribution(m, new Exp(4));
t.setEnablingConditions(m, jobClass, p, 1);
t.setInhibitingConditions(m, jobClass, p, 3);
t.setFiringOutcome(m, jobClass, sink, 1);
```

Markov Chains (solved directly)

```
MarkovProcess ctmc = new MarkovProcess(Q);
Matrix pi = new SolverCTMC(ctmc).getProbSys().probability;
MarkovChain dtmc = new MarkovChain(P);
Matrix piD = new SolverCTMC(dtmc).getProbSys().probability;
Matrix piT = ctmc.transientProb(null, 10.0);
MarkovChain jump = ctmc.toEmbedded();
```

Random Environments

```
Environment env = new Environment("R", 2);
env.addStage(0,"Fast","operational",fastModel);
env.addStage(1,"Slow","degraded",slowModel);
env.addTransition(0,1,new Exp(0.5));
env.addTransition(1,0,new Exp(1.0));
new ENV(env, m -> new FLD(m)).getAvgTable();
```

Finite Capacity Regions

```
Node[] nodes = {queue1, queue2};
FiniteCapacityRegion fcr = model.addRegion(nodes);
fcr.setGlobalMaxJobs(10);
fcr.setClassMaxJobs(class1, 5);
fcr.setDropRule(class1, DropStrategy.Drop);
```

Load-Dependent Service

```
queue.setLoadDependence(alpha); // Matrix
queue.setClassDependence(ni -> Math.min(ni.elementSum(), c),
c);
```

Impatience & Retrials

```
queue.setPatience(jobClass,
ImpatienceType.RENEGING, new Exp(0.1));
queue.setBalking(jobClass, strategy, thresholds);
queue.setRetrial(jobClass, new Exp(2), maxAttempts);
```

State Probabilities

```
solver.getProb(node); // marginal
solver.getProbAggr(node); // aggregate
solver.getProbSys(); // joint
new NC(m).getProbSysMarg(n); // totals
solver.getGenerator(); // CTMC infin.
solver.getStateSpace();
```

Transient & Distributions

```
solver.getTranAvg(); // transient means
solver.getCdfRespT(); // response-time CDF
model.setReward("Cost", state -> ...);
solver.getAvgReward();
```